

ИНТЕЛЛЕКТ ➤

LLM агенты

к.ф.-м.н. Тихомиров Михаил Михайлович

старший научный сотрудник
НИВЦ МГУ имени М. В. Ломоносова

Эволюция LLM: от чат-ботов к агентам



Московский
государственный
университет
имени М.В. Ломоносова



Что такое агентная система?

Две точки зрения



Взгляд со стороны **бизнеса**

- Агент выполняет некоторую бизнес функцию
- Не так важно, что “внутри” системы
- Важна автоматизация
- Цели: экономия, ускорение и (редко) качественное изменение бизнеса

Исследовательский взгляд

- Сложная система на базе LLM и ее взаимодействия со “средой”
- Возникает вопрос автономности в решениях

Что такое агентная система?

Бизнес



- Бизнес всегда хочет зарабатывать деньги
 - Замена людей на автономные системы
 - Ускорение работы людей
 - Повышения качества работы людей
- Агент = бизнес функция, Агент = стероиды для сотрудников
- Часто бывают нереалистичные ожидания от ИИ-агентов
 - Необходимо явно проговаривать ограничения, доступы, возможности
- Большинство внедрений ИИ сейчас “проваливаются”, но это очень временно

Что такое агентная система?

Исследовательский взгляд



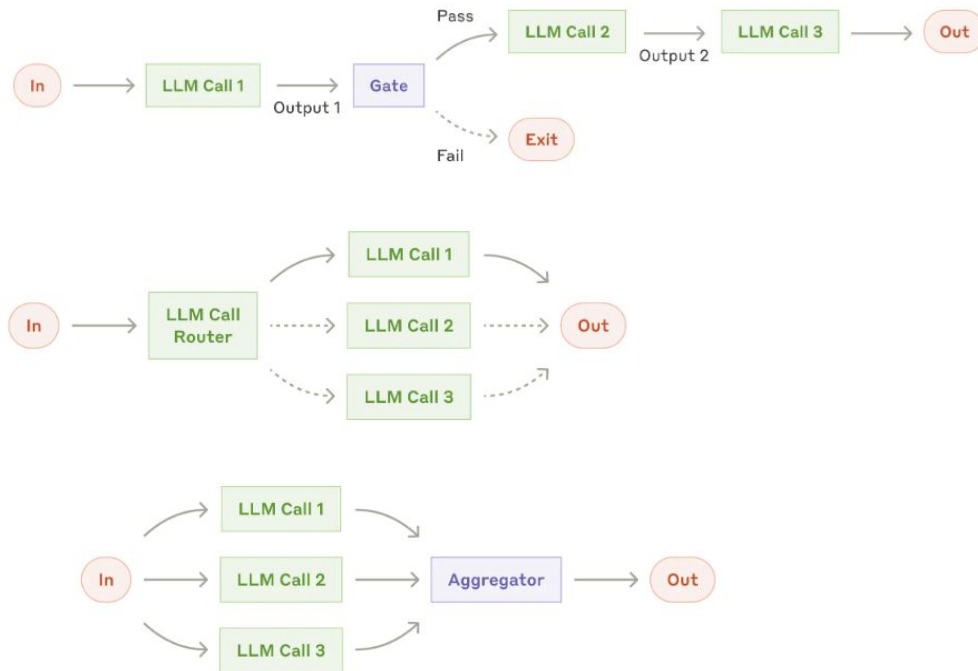
Anthropic разделяют

- **Workflows** - системы на базе LLM, в рамках которой задачи решаются путем исполнения некоторого графа / пути
- **Агенты** - автономные системы, в которых LLM сами определяют то, как нужно решать поставленную задачу на базе использования доступных инструментов

Openai 5 Stages of AI

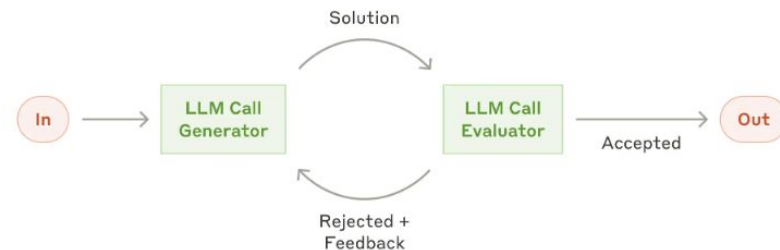
1. **Чат боты** - диалоговые системы
2. **Ризонеры** - модели способные решать сложные задачи
3. **Агенты** - системы, которые могут автономно работать над пользовательскими задачами продолжительное время
4. **ИИ инноваторы** и 5. **ИИ организации**

- Цепочка промптов
- Роутинг
 - Например, классификация запроса на обычный, RAG, неприемлемый -> разные пути исполнения
- Параллелизация
 - Одна и та же задача в разные (или одну) llm -> агрегация, например, голосованием

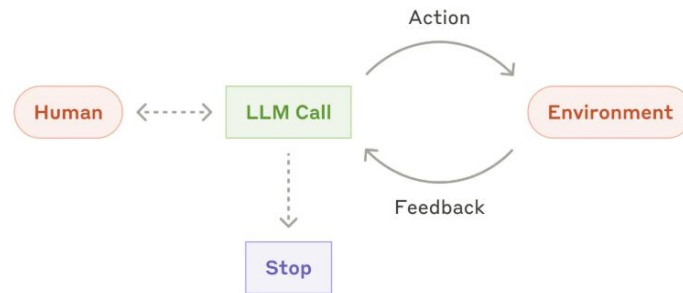


Текущие паттерны уже сложнее отличить от агентов, но важна фиксированная структура графа

- Оркестратор-исполнитель
 - LLM разбивает задачи на подзадачи, отправляет их разным “исполнителям” и синхронизирует результаты
- Оценщик-оптимизатор
 - Решаем задачу, пока не достигнем условия успеха



- Имеется задача от человека, среда, допустимый набор действий
- LLM на базе поступившей задачи взаимодействует со средой через “действия” - вызовы функций (function calling)



В момент t , агент получает наблюдение $o_t \in O$ от среды, затем осуществляет действие $a_t \in A$. Тем самым имеется последовательность $ct = (o_1, a_1, \dots, o_{t-1}, a_{t-1}, o_t)$ - контекст. Модель решает задачу $ct \rightarrow at$ с целью достижения некоторого результата.



- **Низкая автономность**
 - **Human-in-the-loop**, агент генерирует план / код, человек исполняет
- **Средняя автономность**
 - **Human-on-the-loop**, агент работает автономно, но запрашивает approve от пользователя на ключевые действия
- **Высокая автономность**
 - **Human-out-the-loop**, агент получил задачу и исполняет ее. Исполнение “скрыто” от человека
- Чем **выше автономность** - тем **меньше усилий** от человека (при успехе), но **риски** кратно **растут**
- В любом случае необходимо, чтобы человек валидировал итоговый результат агентов в важных задачах и доменах!



Агентная система

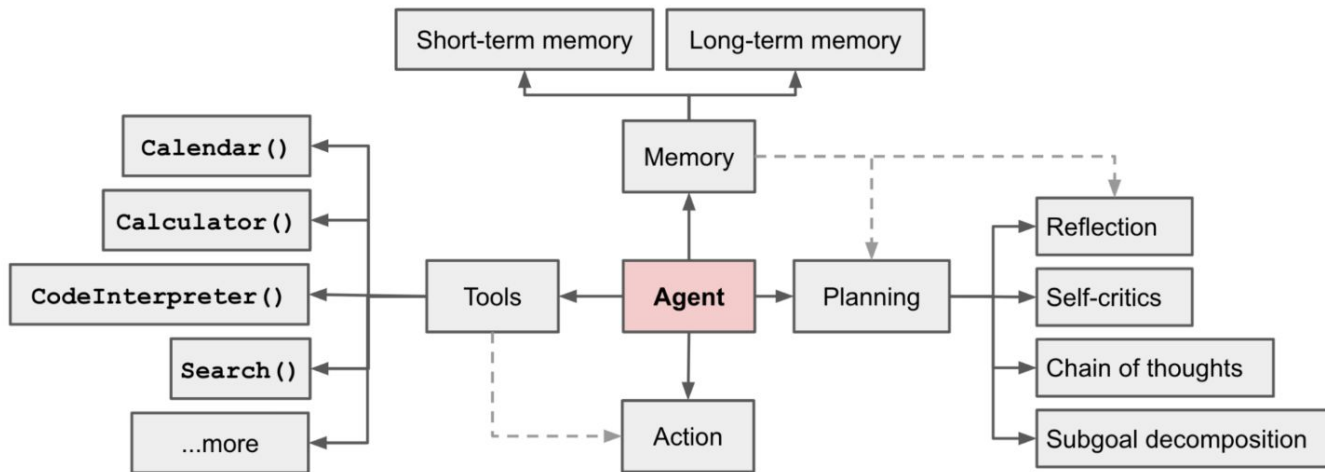
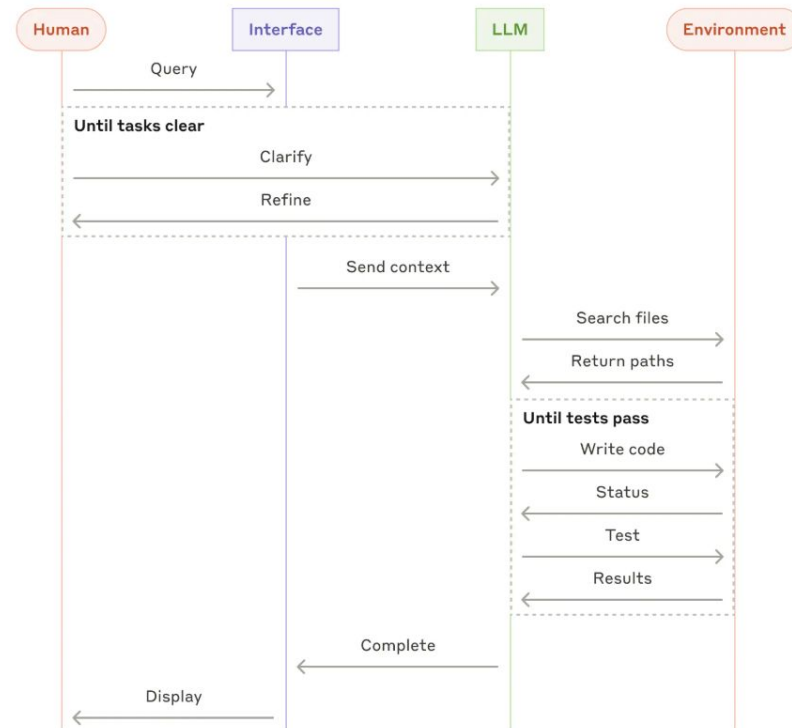


Figure 1: Overview of a LLM-powered autonomous agent system.



Агентная система: среда

- Агент не “видит” и не “находится” в среде, он с ней взаимодействует посредством “действий”
 - tool / function calls
 - Например, bash + команды (ls, grep, python)
 - “Кастомные” команды системы





Идея в разбиении action на 2 шага:

- Размышления (reasoning) - не влияет на среду
- Непосредственно действие (action) - уже возникает взаимодействие со средой, но действие происходит с учетом размышлений

Thought: ...
Action: ...
Observation: ...
... (Repeated many times)



ReAct пример

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod



(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch



(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control ...

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ...

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

Act 4: Finish[yes]



(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys]





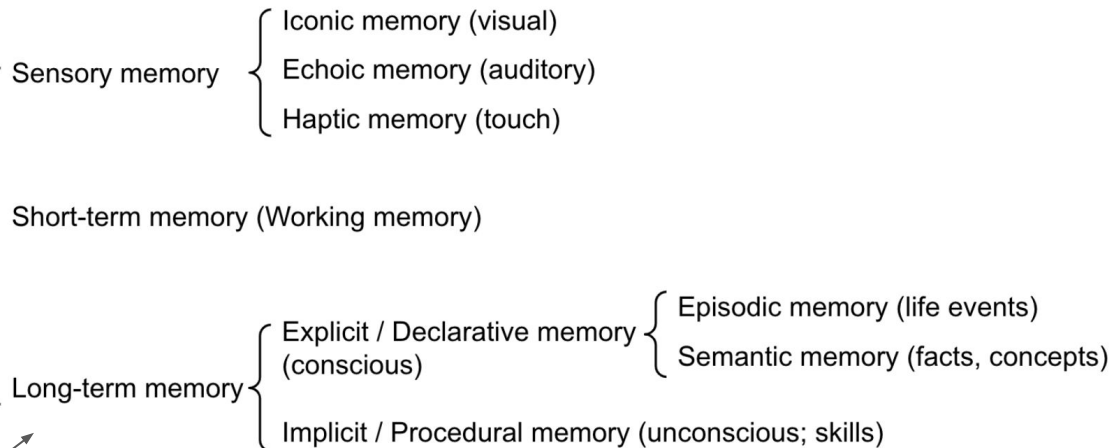
Агентная система: память

Входные эмбединги

Контекст

Внешнее хранилище
памяти (vector db / files / ...)

Memory





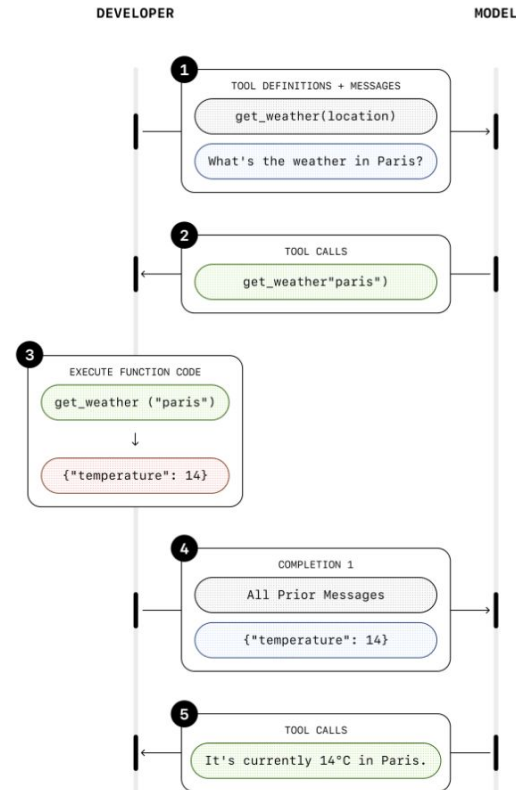
Проблемы памяти в агентных системах

- Система по умолчанию ничего не знает о пользователе: имя, какой формат общения ему приятен и тп
- Для исполнения длинных задач контекста всегда не хватает
 - tool calls быстро наполняют контекст
 - Иногда решают через pruning / summarizing
- В случае агента общего профиля нужно запоминать информацию о пользователе, задачах и тд
- Крайне релевантное исследовательское направление



Агентная система: tools / functions

- Языковая модель - это просто набор весов, она не может “исполнять”, она может только предсказывать следующий токен, генерировать текст
- LLM может сгенерировать запрос, например, “вызови функцию `get_weather` и верни мне результат”
- На основе результата вызова - ответ пользователю
- Для этого есть спец. формат + этим навыкам LLM надо учить на post-train этапе





Немного истории: toolformer

Проблема:

- Языковая модель плохо справляется с вычислениями, фактами и тп.

Решение:

- Можно научить ее использовать внешние инструменты (API) для этих целей
 - Калькулятор, внешние бд и тп.

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

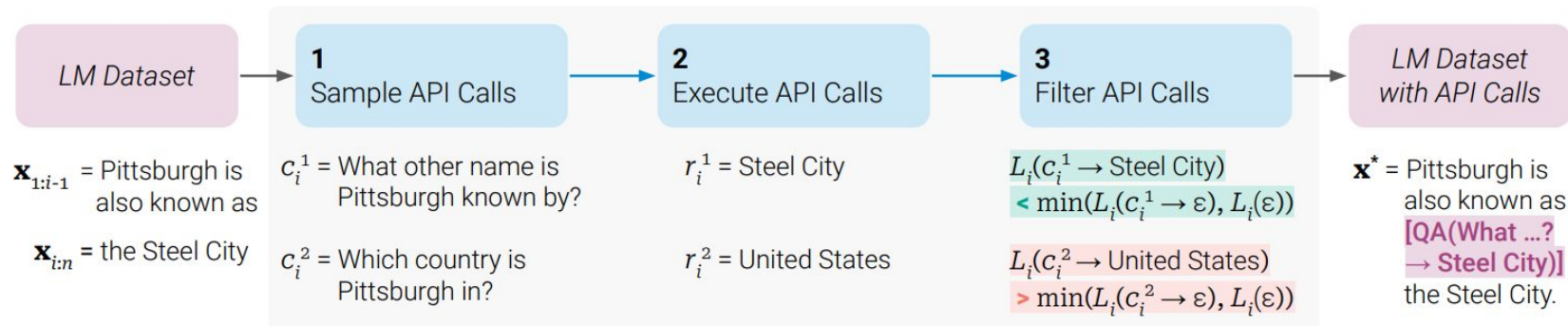
The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.



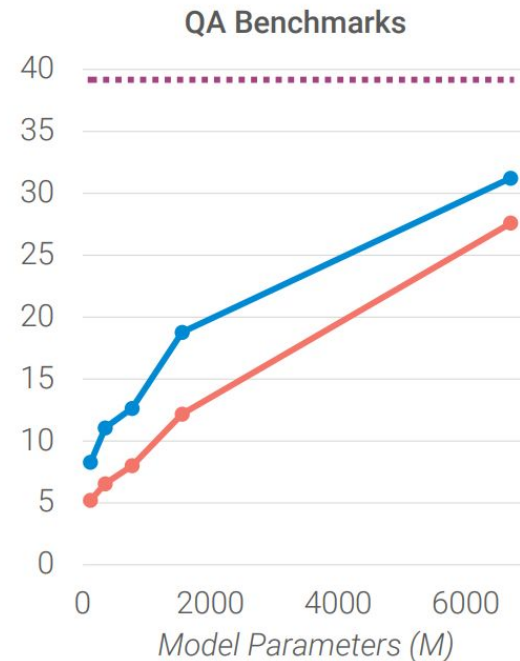
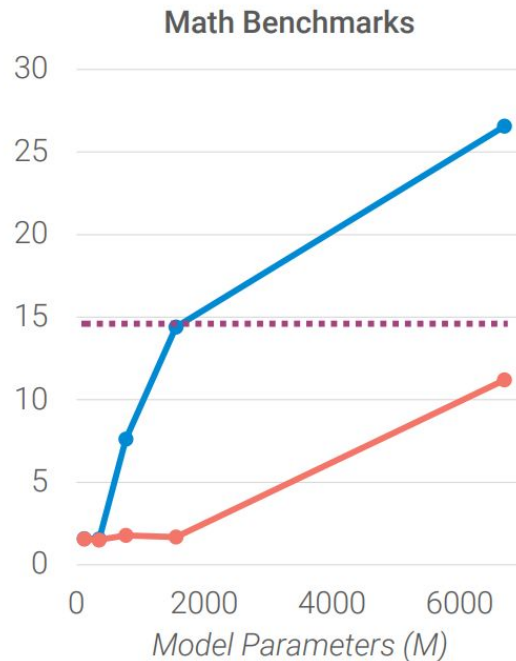
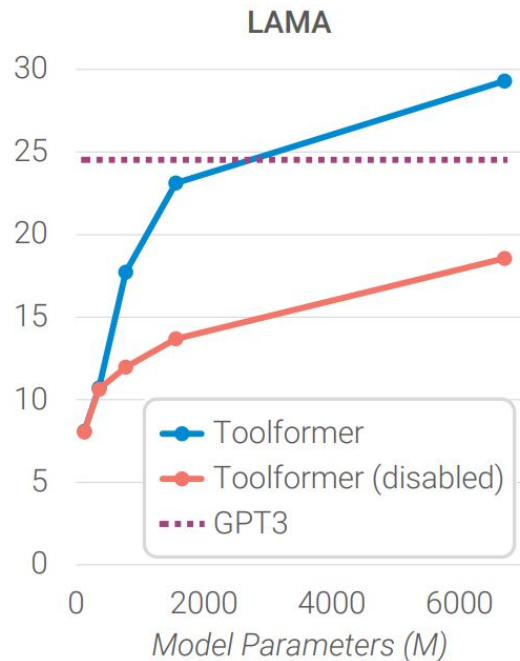
Toolformer - как обучается

- Используя **few-shot** (обучение в контексте) и фильтрацию, сгенерировать датасет с примерами API вызовов,
- Обучить модель (SFT) на полученном датасете,
- Во время использования (inference) при генерации специального токена, сигнализирующего об API вызове, прерывать декодирование и осуществлять вызов.





Toolformer - результаты





Messages с tool calls

- Система отслеживает наличие tool_calls в ответе модели
- Если они есть, то эти функции вызываются на стороне программы, не llm!
- Результат возвращается со специальной ролью tool

```
[
  {
    "role": "system",
    "content": "Ты ИИ Агент"
  },
  {
    "role": "user",
    "content": "Какая сегодня погода в Париже?"
  },
  {
    "role": "assistant",
    "content": null,
    "tool_calls": [
      {
        "id": "chatcmpltoola5de2a8b6ed5466d93f883c2ad06",
        "type": "function",
        "function": {
          "name": "get_weather",
          "arguments": "{\"location\": \"Париж\"}"
        }
      }
    ]
  },
  {
    "role": "tool",
    "content": "14",
    "tool_call_id": "chatcmpltoola5de2a8b6ed5466d93f883c2ad06"
  },
  {
    "role": "assistant",
    "content": "Сегодня в Париже 14 по Цельсию"
  }
]
```



Structured output

- Вероятности недопустимых по схеме в текущий момент токенов -> 0
- Нужен конкретный json формат? -> обязательно к использованию
- На базе SO можно реализовывать более надежные инструменты

```
from pydantic import BaseModel
from openai import OpenAI

class Info(BaseModel):
    name: str
    age: int

client = OpenAI(base_url="http://0.0.0.0:8000/v1", api_key="dummy")
completion = client.beta.chat.completions.parse(
    model="meta-llama/llama-3.1-8B-Instruct",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "My name is Cameron, I'm 28. What's my name and age?"},
    ],
    response_format=Info,
    extra_body=dict(guided_decoding_backend="outlines"),
)

message = completion.choices[0].message
print(message)
assert message.parsed
print("Name:", message.parsed.name)
print("Age:", message.parsed.age)
```

Output:

```
onMessage[Testing](content='{"name": "Cameron", "age": 28}', refusal=None, role='assistant', audio=None
```



Как вызвать модель с tool calls

- При этом в инференс движке может быть реализован **structured output** применительно к tool calls -> модель не может ошибиться в генерируемой сигнатуре

```
from openai import OpenAI
import json

client = OpenAI(base_url="http://localhost:8000/v1", api_key="dummy")

def get_weather(location: str, unit: str):
    return f"Getting the weather for {location} in {unit}..."
tool_functions = {"get_weather": get_weather}

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get the current weather in a given location",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {"type": "string", "description": "City and state, e.g., 'San Francisco'"},
                    "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
                },
                "required": ["location", "unit"],
            },
        },
    },
]

response = client.chat.completions.create(
    model=client.models.list().data[0].id,
    messages=[{"role": "user", "content": "What's the weather like in San Francisco?"}],
    tools=tools,
    tool_choice="auto",
)

tool_call = response.choices[0].message.tool_calls[0].function
print(f"Function called: {tool_call.name}")
print(f"Arguments: {tool_call.arguments}")
print(f"Result: {tool_functions[tool_call.name](**json.loads(tool_call.arguments))}")
```



Как модель видит tool calls?

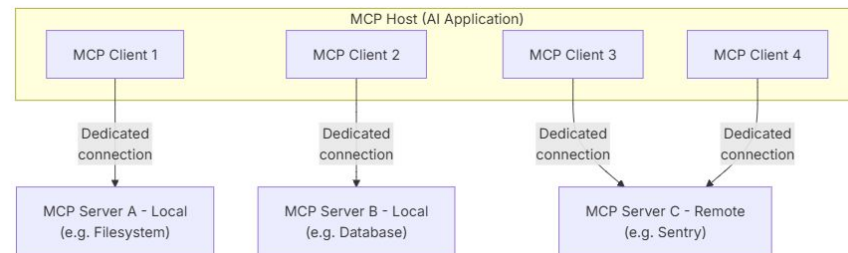
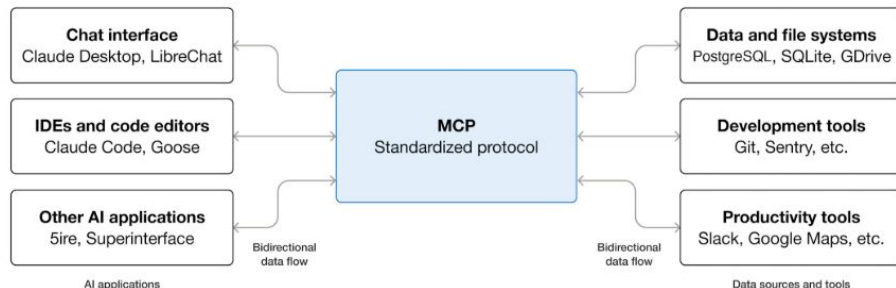
- В каждой модели свой chat template
- Чаще всего подкладывается в system часть промта
- Пример для Qwen3.5 ->

```
{%- if tools and tools is iterable and tools is not mapping %}
{{- '<|im_start|>system\n' }}
{{- "# Tools\n\nYou have access to the following functions:\n\n<tools>" }}
{%- for tool in tools %}
{{- "\n" }}
{{- tool | tojson }}
{%- endfor %}
{{- "\n</tools>" }}
{{- '\n\nIf you choose to call a function ONLY reply in the following format with'
{%- if messages[0].role == 'system' %}
{%- set content = render_content(messages[0].content, false, true)|trim %}
{%- if content %}
{{- '\n\n' + content }}
{%- endif %}
{%- endif %}
{{- '<|im_end|>\n' }}
```



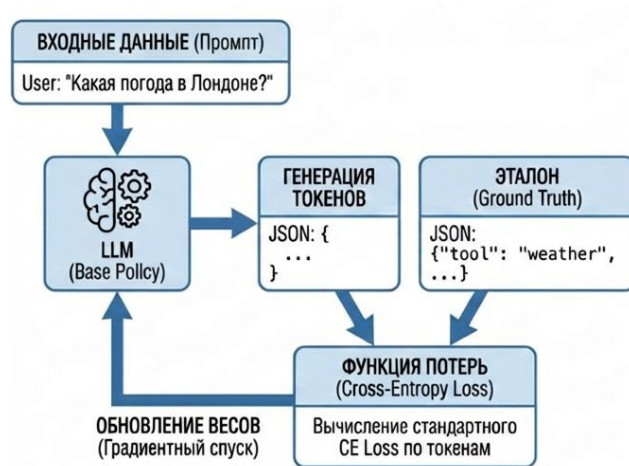

Model Context Protocol (MCP)

- Проблема масштабируемости и унификации - разные системы, разные tools, разные форматы
- MCP - протокол взаимодействия агентов с инструментами, унификация процесса
- Похоже на REST API



Как LLM учат использовать инструменты

- Этап 1: SFT - учим формату. Идеино аналогично toolformer, однако в современных реалиях
- Этап 2: RL - учим модель выбирать, применять tools для достижения результата. Правильных траекторий множество, нельзя учить только SFT



Задача: Научиться правильно 'писать' вызов инструмента

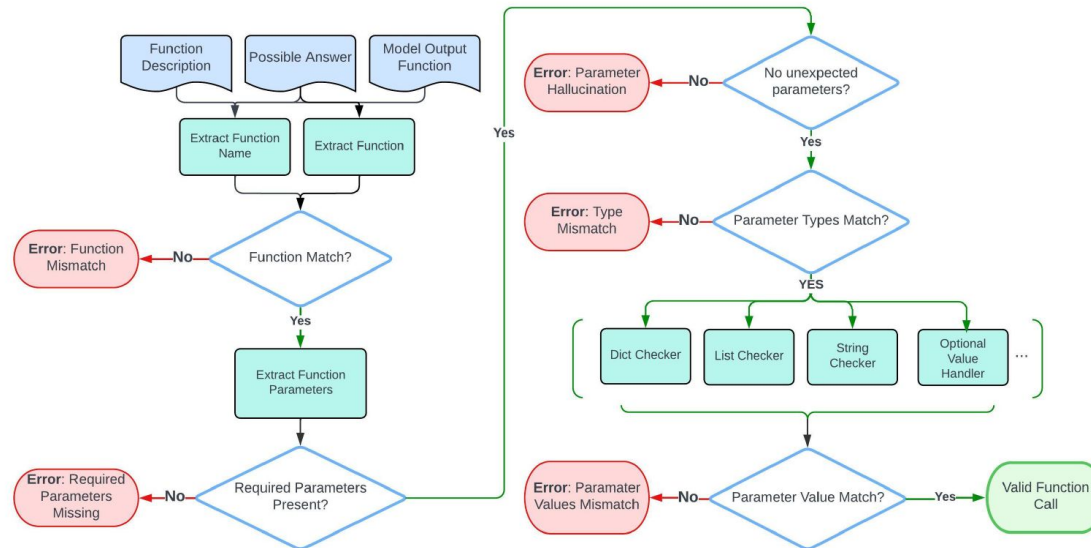


Задача: Найти лучший путь к решению через пробы и награды



Berkeley Function-Calling Leaderboard

- Измерялась способность моделей по запросу пользователя и документации набора функций правильно выбирать функцию и параметры





Berkeley Function-Calling Leaderboard

BFCL: From Tool Use to Agentic Evaluation of Large Language Models

The Berkeley Function Calling Leaderboard (BFCL) V4 evaluates the LLM's ability to call functions (aka tools) accurately. This leaderboard consists of real-world data and will be updated periodically. For more information on the evaluation dataset and methodology, please refer to our blogs: [BFCL-v1](#) introducing AST as an evaluation metric, [BFCL-v2](#) introducing enterprise and OSS-contributed functions, [BFCL-v3](#) introducing multi-turn interactions, and [BFCL-v4](#) introducing holistic agentic evaluation. Checkout [code](#) and [data](#).

Last Updated: 2026-04-12 [\[Change Log\]](#)

Search

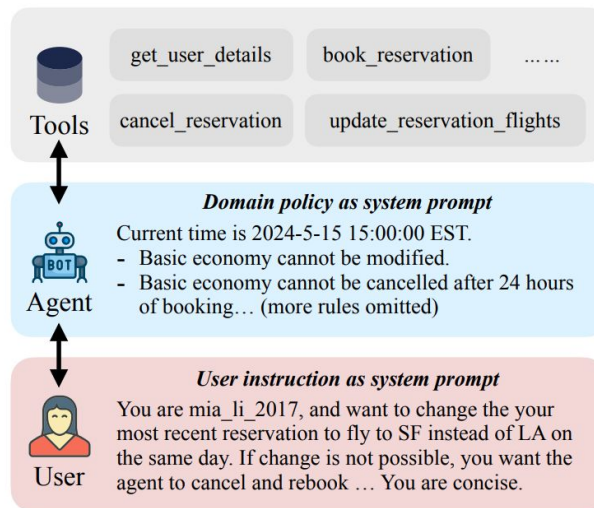
			Agentic			Multi Turn	Single Turn		Hallucination Measurement		Format Sensitivity		Latency (s)		
Rank	Overall Acc	Model	Cost (\$)	Web Search ▶	Memory ▶	Multi turn ▶	Non-live (AST) ▶	Live (AST) ▶	Relevance	Irrelevance	Max Delta	SD	Mean	Organization	Location
				Overall Acc	Overall Acc	Overall Acc	Overall Acc	Overall Acc							
1	77.47	Claude-Opus-4-5-20251101 (FC)	86.55	84.5	73.76	68.38	88.58	79.79	62.5	84.72	N/A	N/A	4.38	Anthropic	Proprietary
2	73.24	Claude-Sonnet-4-5-20250929 (FC)	43.73	81	64.95	61.37	88.65	81.13	68.75	86.61	N/A	N/A	4.31	Anthropic	Proprietary
3	72.51	Gemini-3-Pro-Preview (Prompt)	298.47	80	61.72	60.75	90.65	83.12	68.75	85.59	8.5	1.7	12.08	Google	Proprietary
4	72.38	GLM-4.6 (FC thinking)	4.64	77.5	55.7	68	87.56	80.9	75	84.96	N/A	N/A	4.34	Zhipu AI	Mainland China
5	69.57	Grok-4-1-fast-reasoning (FC)	17.26	82.5	53.98	58.87	88.27	78.46	81.25	79.43	N/A	N/A	6.74	xAI	Proprietary
6	68.7	Claude-Haiku-4-5-20251001	14.23	83.5	54.41	53.62	86.5	78.68	62.5	85.11	N/A	N/A	1.68	Anthropic	Proprietary



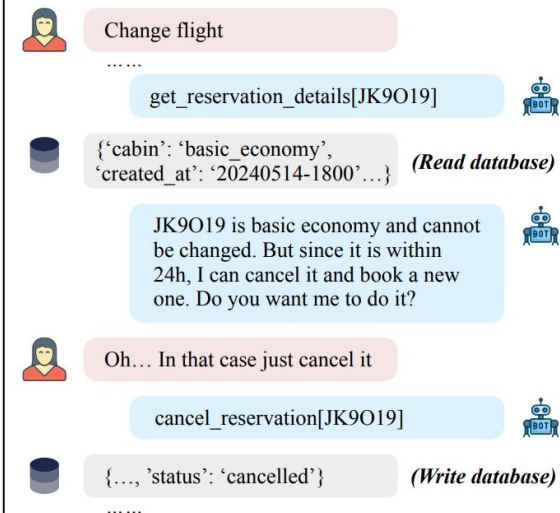
Tau-bench

- Измеряет агентные способности
- LM-simulated users
- Метрика pass^k - все К запусков должны быть успешны

(a) τ -bench setup



(b) Example trajectory in τ -airline





Tau-bench: результаты

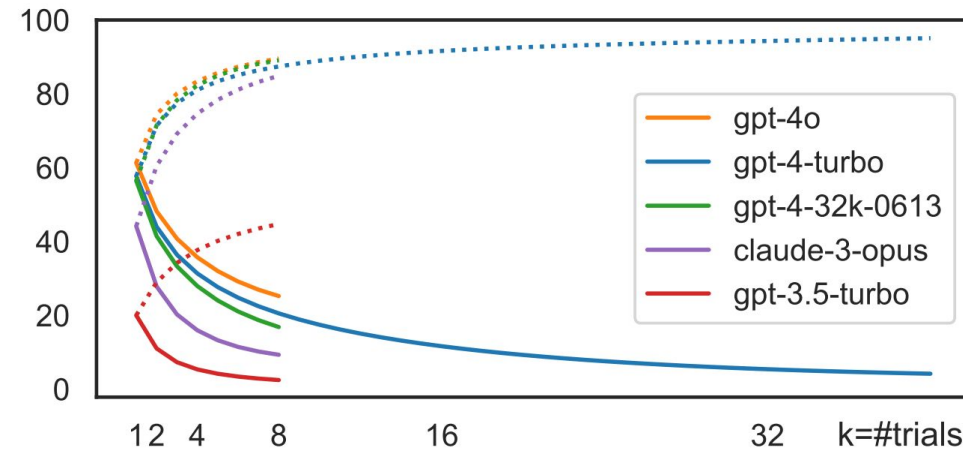


Figure 4: pass^k (—) and pass@k (..) in τ -retail.

τ -bench Leaderboard

Overall Banking Retail Airline Telecom Standard Custom Legacy (v1)

RANK	MODEL	RELEASED	SUBMITTING ORG	REASONING	USER SIM	PASS^1	PASS^2	PASS^3	PASS^4
#1	Qwen3.5-397B-A17B	Feb 16, 2026	Sierra	Enabled	gpt-5.2				56.2%
#2	Claude Opus 4.5	Nov 24, 2025	Sierra	High	gpt-5.2				52.3%
#3	GPT-5.2	Dec 11, 2025	Sierra	High	gpt-5.2				51.2%
#4	Gemini 3 Flash	Dec 17, 2025	Sierra	High	gpt-5.2				48.5%
#5	Gemini 3 Pro	Nov 18, 2025	Sierra	High	gpt-5.2				48.0%
#6	GLM-5	Feb 11, 2026	Sierra	Enabled	gpt-5.2				44.8%
#7	Claude Sonnet 4.5	Sep 29, 2025	Sierra	Enabled	gpt-5.2				40.5%
#8	GPT-5.2	Dec 11, 2025	Sierra	None	gpt-5.2				31.8%



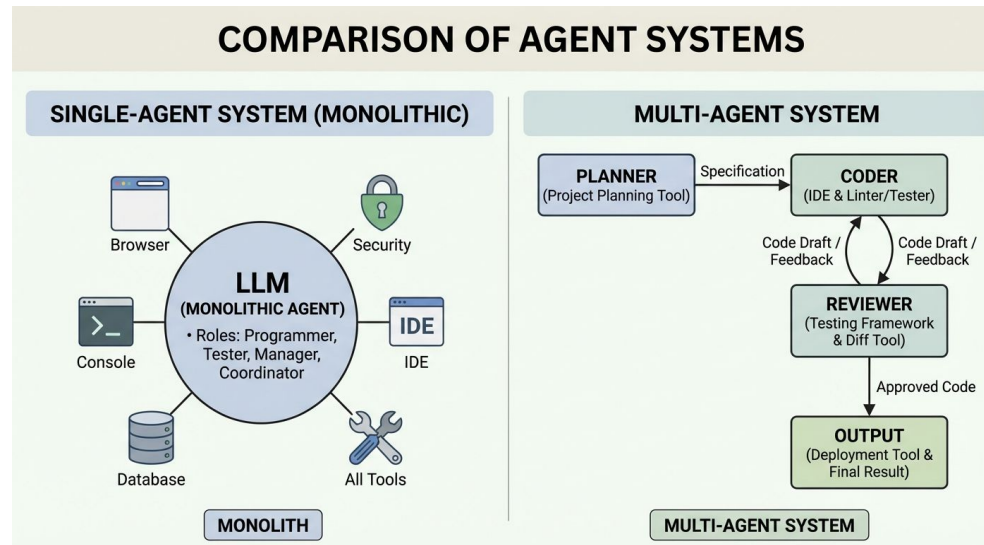
Мультиагентные системы

Single agent

- Один контекст
- Единый набор правил на все случаи жизни
- Аналогия: программа - монолит

Multi agent system

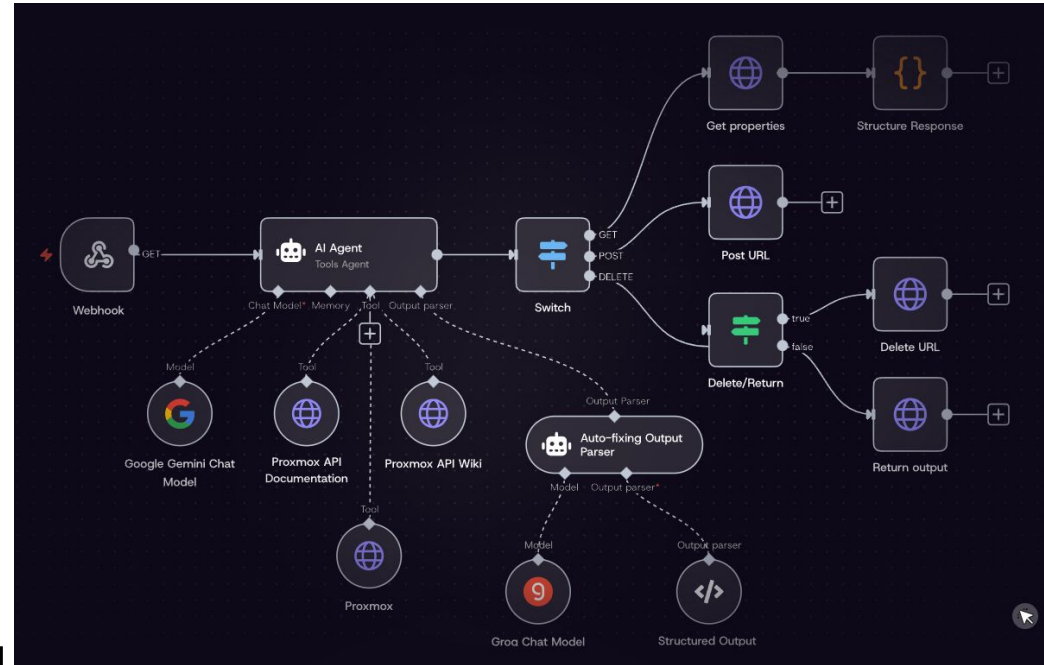
- Разделение обязанностей -> меньше лишней информации в контексте -> лучше результат
- Возможны разные паттерны взаимодействия:
 - Оркестратор, планировщик, исполнитель, критик / ревьювер, тестировщик





Low-Code среды: n8n

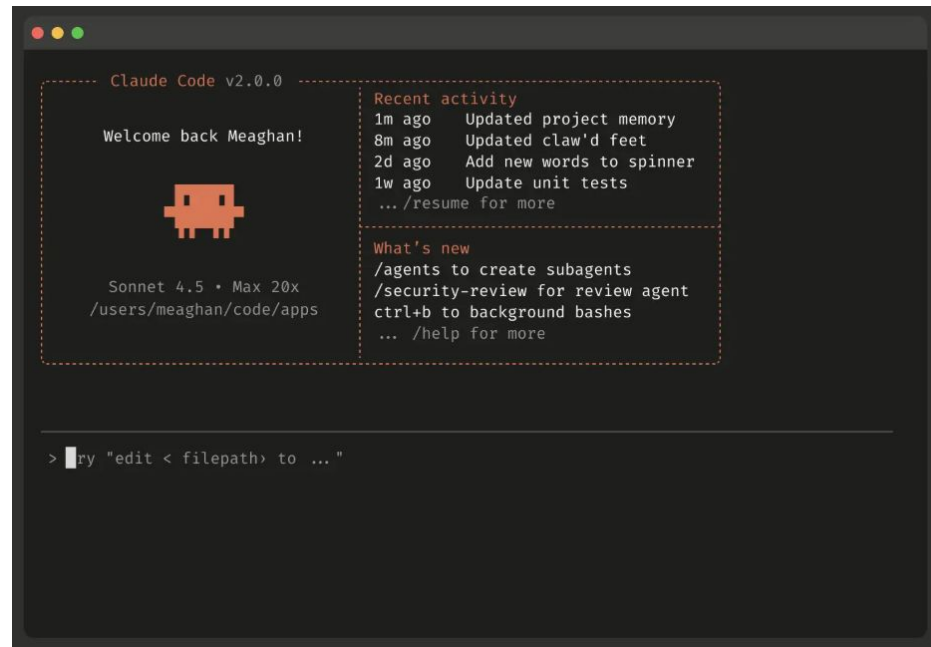
- Популярный открытый фреймворк
- Может быть развернут локально
- Множество готовых модулей и интеграций
- Создание агентов в low / zero код режиме
- Особенно удобно для workflow систем
- Полезно для прототипирования / MVP / POC





Кодинг-агенты

- Специализированы на задачи разработки
- Могут работать в больших проектах (не читают всё в память за раз)
- Настраиваются через специальные файлы, например, CLAUDE.md
 - Но это по сути часть промпта, а не железные правила
- Основа разработки ПО сейчас и в будущем



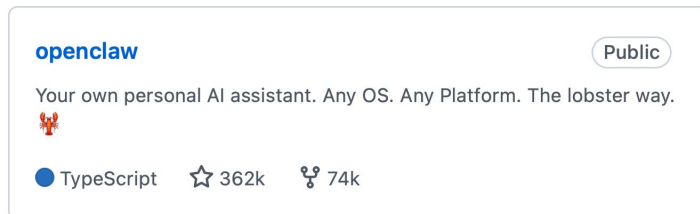
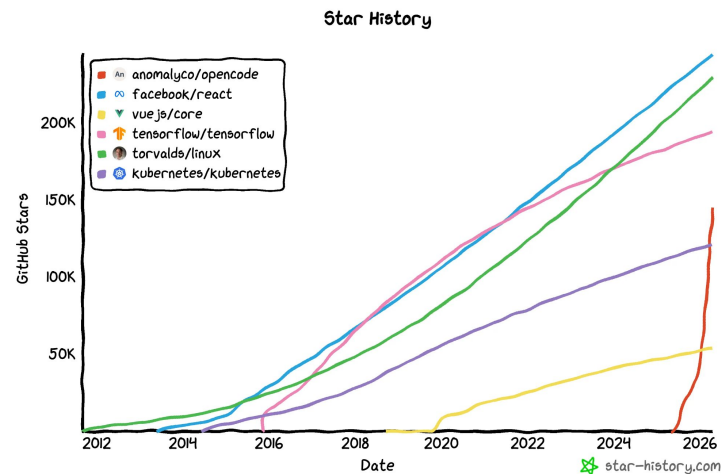
Платформа для создания
персональных ИИ-ассистентов на
основе популярных LLM

Феноменальная популярность:
362.000 звезд на GitHub; 74.000 fork

3.2 миллиона активных
пользователей

И все за 5 месяцев!

Стал частью OpenAI, но код под
лицензией MIT



Что умеет OpenClaw?



- **Интеграция с мессенджерами:** WhatsApp, Telegram, Slack, Discord, Google Chat, Signal, iMessage → Можно общаться с агентом как с человеком
- Работает поверх **любой** желаемой **LLM**, нативная интеграция с Claude, GPT, Gemini, DeepSeek, Ollama и тд
- Может вызывать специализированных кодировщиков (claude code и др)
- Может **выполнять** Shell команды и **взаимодействовать** с локальными файлами
- **Хранит историю взаимодействия** между сессиями для большей персонализации



Сравнение OpenClaw с традиционными ИИ-интерфейсами

Характеристика	Традиционные чат-боты (ChatGPT/Claude/Qwen/DeepSeek)	OpenClaw
Режим работы	Реактивный (ждет промпта и отвечает в рамках сеанса)	Проактивный (фоновые задачи)
Доступ к системе	Отсутствует (ограничен песочницей чата)	Полный доступ (Shell, файлы, браузер)
Хранение данных	Облако провайдера модели	Локальные Markdown-файлы
Интерфейс	Веб-сайт/Приложение	Мессенджеры (WhatsApp/Telegram)
Управление	Единичный сеанс	Постоянная память и навыки



Архитектура OpenClaw

1. Оркестратор (Gateway)

Детерминированный контроль и маршрутизация

Принимает сообщения из мессенджеров, собирает контекст, запускает инструменты (Shell, API)

2. LLM-ядро (Мозг)

Планирование, reasoning, выбор инструмента, интерпретация наблюдений

3. Память и локальные артефакты

Markdown-файлы, заметки и долгоживущее состояние задачи

4. Runtime с инструментами

Shell, чтение файлов, браузер, API и другие действия во внешнем мире

5. Интерфейс пользователя

Мессенджеры и событийный контур, где агент получает новые триггеры



OpenClaw не использует БД, вся личность и правила хранятся в текстовых файлах.

- **SOUL.md**: Личность, тон общения и этические ограничения агента
- **AGENTS.md**: Операционные правила и SOP (стандартные процедуры выполнения задач)
- **TOOLS.md**: Карта возможностей; описание специфичных для данного сервера инструментов
- **MEMORY.md**: Отобранные знания о пользователе и долгосрочные факты
- **IDENTITY.md**: Метаданные агента (имя, ID, роль для маршрутизации)

Трехуровневая система расширений: Tools, Skills, Plugins



- **Tools:** Низкоуровневые функции для работы с файлами, сетевыми запросами, браузером и другими базовыми действиями. Это «руки» агента.
- **Skills:** Markdown-файлы формата SKILL.md. Обычно задают поведение: когда вызывать навык, какие шаги делать, какие ограничения соблюдать.
- **Plugins:** npm-пакеты для тяжелой или внешней интеграции: API, сложные workflow, доменная бизнес-логика

```
---
name: morning-briefing
trigger: cron 0 7 * * *
---

# Morning Briefing
Every morning at 7 AM:
1. Check unread emails (last 12 hours)
2. Summarize any calendar conflicts today
3. List open GitHub PRs that need review
4. Check weather for the day
5. Send summary to Telegram
Keep it under 200 words. No fluff. Lead with anything urgent.
```



Проактивное поведение

- Поддержка cron задача - планирование действий по расписанию
- **Heartbeat:** Проверка системы каждые N минут через HEARTBEAT.md.
Пользователь сам задает раз во сколько минут нужно исследовать систему на наличие обновлений.
- **Автономное принятие решений:** Агент сам решает, нужно ли уведомлять пользователя



- **ClawJacked (CVE-2026-25593):** Уязвимость WebSockets, позволявшая вредоносному сайту в браузере пользователя тайно подключиться к локальному Gateway и перехватывать управление агентом
- **Вредоносные плагины (ClawHavoc):** 341 плагин на ClawHub, которые маскировались под полезные утилиты.
 - **Пример:** Плагин **google-qx4** требовал установки «обязательной утилиты» **openclaw-core**, которая использовалась для кражи данных
- **Leaky Skills:** Популярны плагины (например, **moltyverse-email**), которые по ошибке выводили API-ключи пользователя в открытый лог чата или сохраняли их в незашифрованном виде в **MEMORY.md**

Для защиты ресурсов предложены следующие меры предосторожностей:

- **Изоляция Gateway (Token Isolation):**
 - Запуск Gateway в отдельном Docker-контейнере.
 - Агент не имеет доступа к **.env** файлу с токенами LLM-провайдеров. Gateway сам подставляет ключи при обращении к API.
- **Rootless запуск:**
 - Использование Rootless Docker для запуска процессов от имени обычного пользователя
- **Сетевой Zero Trust:**
 - Привязка Gateway только к **127.0.0.1** (loopback).
 - Доступ извне только через защищенные туннели (Tailscale/CloudConnexa).
- **Делайте дампы:**
 - Не давайте доступ к информации, удаление которой является катастрофой
- **Ограничивайте бюджет на api токен:**
 - Плохо настроенная система может потратить сотни долларов незаметно.



Проблемы OpenClaw

- **Поиск по памяти как RAG:** Проблема качественного Retrieval и Ranking в Markdown-файлах
- **Риски Open Source:** Открытый код и реестры (ClawHub) упрощают распространение вредоносных плагинов (до 20% реестра)
- **Новые метрики успеха:** Переход от точности текста к агентским показателям:
 - End-to-end успех (выполнение задачи целиком)
 - Число шагов до результата (эффективность пути)
 - Recovery rate (способность агента исправиться после ошибки)
 - Стоимость выполнения (Token-per-Task)
- **Необходимость изоляции** через Docker и Zero Trust как стандарт
- **Нестабильность версий:** периодически обновления, которые выходят почти каждый день, ломают настроенную систему



Популярное железо для запуска OpenClaw

Бюджетный (Raspberry Pi 5, 8GB RAM):

- Подходит для легкой автоматизации и мессенджеров
- Стоит использование NVMe SSD вместо SD-карт (SD-карты «умирают» через недели из-за постоянных перезаписей логов)

Оптимальный (Mac Mini M4, 16-32GB RAM):

- «Золотой стандарт» сообщества. Unified Memory устраняет bottleneck между CPU-GPU
- Может одновременно поддерживать 5–10 агентов и браузерную автоматизацию
- С 24GB+ RAM можно локально инферить 7B модель

Локальный инференс (RTX 4090, 64-128GB RAM):

- Для запуска локальных моделей без облачных API
- Такие модели не подходят для сложных задач



Перспективы на ближайшие годы

- **Навыки агентов** будут расти, а инфраструктура развиваться
- Попытки сэкономить будут приводить к высоким рискам -> часто workflow надежнее
- **Агенты будут** становится такими же **необходимыми** системами, как компилятор
- **Новые типы** агентных систем для:
 - Научной деятельности
 - Образовательного процесса
 - Повседневных нужд
- **Всё меняется очень быстро**, LLM по сути могут привести к новой “индустриальной революции”



Выводы

- Ответ на вопрос “**что такое агент**” зависит от контекста самого вопроса и того, кто вопрос задает
- **Основа агентных систем:**
 - LLM + tool / function calling + agentic loop + state + memory
- **Function Calling и Structured Output - технический базис!**
- **RL и среды** - главные инструменты для повышения агентных навыков
 - Как построить правильную среду огромный вопрос
 - Очень свежее исследовательское направление
- Уже **необходимы инженеры**, которые умеют правильно **проектировать агентные системы**
 - Что такое правильно - это открытый вопрос.